



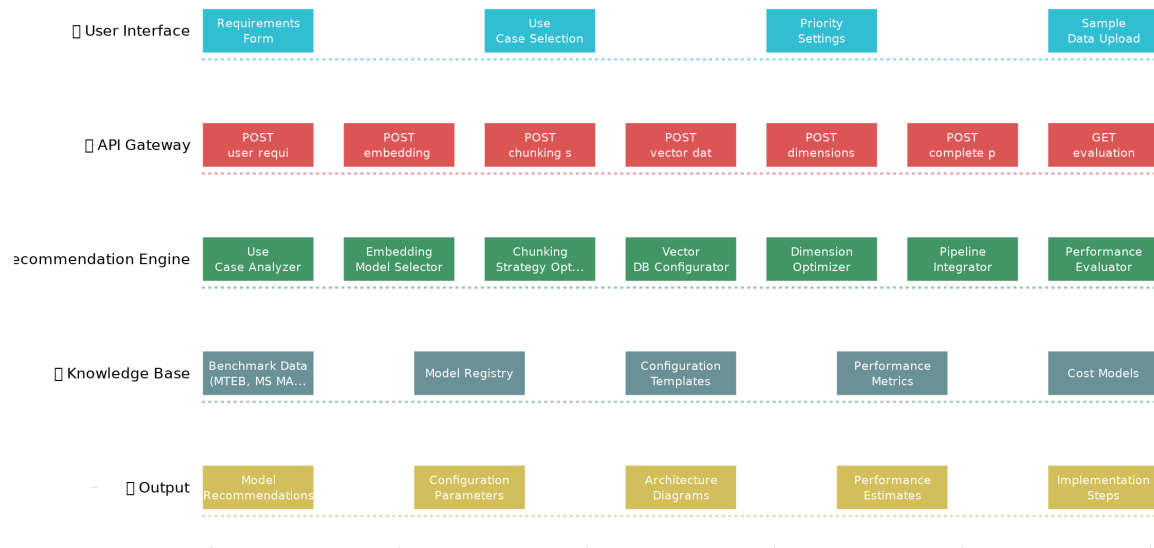
## RAG Pipeline Evaluation Module - Redesigned System

Based on your request to review and recreate the design, I've redesigned the "Design evaluation for RAG" project as a comprehensive **RAG Evaluation Module**. This module focuses on evaluating and recommending optimal parameters for RAG ingestion and retrieval pipelines through static (pre-defined, rule-based) and dynamic (iterative, data-driven) evaluation approaches. It helps users understand and configure pipelines for best outcomes, considering trade-offs in accuracy, latency, cost, and more.

The redesign incorporates:

- **Static Evaluation:** Rule-based logic using benchmarks (e.g., MTEB leaderboard, VectorDBBench) and predefined heuristics without runtime testing. [\[1\]](#) [\[2\]](#) [\[3\]](#) [\[4\]](#) [\[5\]](#)
- **Dynamic Evaluation:** Iterative testing on user-provided sample data, using metrics like ROUGE, BLEU, faithfulness scores, and custom RAG metrics (e.g., retrieval precision@K, answer relevance) with multiple iterations (3-5 cycles) to optimize parameters.
- **Iterative Optimization Logic:** For each component, perform 3-5 iterations: Start with baseline params, measure metrics, adjust (e.g., via grid search or Bayesian optimization), re-evaluate, and converge on optima.
- **API Structure:** 8 specialized APIs for each point, plus an overarching Recommendation API that orchestrates them.
- **User Inputs:** A detailed set of requirements gathered via a central input API.
- **Metrics Integration:** ROUGE/BLEU for generation quality, precision/recall for retrieval, latency measurements for performance.
- **Examples:** Provided for each evaluation.
- **Diagrams:** Generated for design, architecture, flow, API systems, low-level logic, and evaluation processes.

# RAG Pipeline System Architecture



## RAG Pipeline Recommendation System Architecture

### User Input Requirements

To perform evaluations, the system requires detailed user inputs via a central `POST /user-inputs` API. This generates a unique `evaluation_id` used across all APIs. Inputs influence recommendations (e.g., high accuracy priority favors larger models, data privacy prefers on-premise DBs).

**Detailed Input Fields** (not limited to these; expandable based on use case):

- Source Data Type:** e.g., PDF, Word, TXT, RDBMS tables, NoSQL documents, CSV, images. [\[6\]](#) [\[7\]](#)
- Sample Data:** Upload file(s) or text snippets (1-5 representative samples, up to 10MB) for dynamic evaluation.
- RAG Use Case Details:** e.g., "document QA", "knowledge base", "customer support chatbot", "research assistant". [\[8\]](#) [\[6\]](#)
- Volume of Data:** Current total (e.g., 1GB), expected growth (e.g., 10% monthly).
- Cost Constraints:** Budget per month (e.g., <\$100), optional but influences cost-optimized recommendations.
- Priorities:** Ranked (e.g., 1. Accuracy, 2. Latency, 3. Cost) on a 1-10 scale.
- Data Privacy Requirements:** e.g., "GDPR-compliant", "on-premise only", "cloud with encryption".
- Complexity of Queries:** e.g., "simple keyword", "complex multi-hop reasoning".

9. **Latency Tolerance:** e.g., "<200ms per query".
10. **Accuracy Tolerance:** e.g., ">90% retrieval precision".
11. **Truthfulness Tolerance:** e.g., "High (minimal hallucinations)".
12. **Additional Factors:** Query volume (e.g., 1000/day), domain (e.g., medical, legal), languages, hardware constraints (e.g., GPU availability).

#### API Input Schema Example:

```
{
  "use_case": "document QA",
  "priorities": {"accuracy": 9, "latency": 7, "cost": 5},
  "data_source": {"type": "PDF", "volume_gb": 5, "growth_rate": "10% monthly"},
  "sample_data": ["base64_encoded_file_or_text"],
  "privacy_level": "high",
  "query_complexity": "medium",
  "latency_ms": 300,
  "accuracy_threshold": 0.9,
  "truthfulness_threshold": 0.95
}
```

**Output:** {"evaluation\_id": "uuid", "validation\_status": "ready"}.

## API Structure and Endpoints

All APIs are under `/api/v1/rag-evaluation`. Each of the 8 APIs has input (includes `evaluation_id`, sample data subset) and output schemas. They support static (quick, no iterations) and dynamic (iterative testing) modes via a query param (e.g., `?mode=dynamic&iterations=5`).

### 1. Embedding Model API (POST `/embedding-model`)

**Purpose:** Recommends embedding models with params (dimensions, batch size, normalization, GPU use, half precision, `num_workers`).<sup>[1] [2] [9]</sup>

- **Static Evaluation Logic:** Use MTEB scores and heuristics (e.g., if accuracy priority >8, select high-MTEB models like text-embedding-3-large; factor in cost/latency).
- **Dynamic Evaluation Logic:** 3-5 iterations: Embed sample data with candidate models, measure semantic similarity (cosine), ROUGE with ground truth, adjust params (e.g., reduce batch if latency high), select best via weighted score ( $0.6accuracy + 0.3latency + 0.1cost$ ).
- **Iteration Logic:** Start with defaults, evaluate metrics, use grid search to tweak batch/GPU, repeat until convergence (<5% improvement).
- **Input:** `evaluation_id`, `mode`, optional candidates.
- **Output:** {"recommended\_model": "text-embedding-3-large", "params": {"dimensions": 3072, "batch\_size": 32, "gpu": true, ...}, "metrics": {"mteb\_score": 0.85, "latency\_ms": 150}}.
- **Example:** For medical QA use case with high accuracy, static recommends text-embedding-3-large (MTEB: 0.82); dynamic tests on samples, iterates batch from 16-64, selects 32 for

92% ROUGE.

## 2. Chunking Strategy API (POST /chunking-strategy)

**Purpose:** Recommends strategy (e.g., recursive, semantic) with params (size, overlap, delimiter, combine\_small\_chunks, min\_chunk\_size, normalization). [\[10\]](#) [\[11\]](#) [\[12\]](#) [\[13\]](#)

- **Static Evaluation Logic:** Rule-based on data type (e.g., PDF → semantic chunking with 1000 size, 200 overlap if structured).
- **Dynamic Evaluation Logic:** 4 iterations: Chunk samples, embed/retrieve, measure ROUGE/BLEU vs full text, adjust overlap/size (e.g., +50 if ROUGE < 0.8), optimize for context preservation.
- **Iteration Logic:** Baseline chunk, evaluate fragmentation (chunk count vs info loss), gradient descent-like adjustment, stop at metric plateau.
- **Input/Output:** Similar to above, with "strategy": "semantic", "params": {"size": 1000, "overlap": 200, ...}.
- **Example:** For TXT narratives, static suggests recursive (size=800, overlap=150); dynamic iterates size 600-1200, selects 1000 for 88% BLEU.

## 3. Dimensions Optimization API (POST /dimensions-optimization)

**Purpose:** Recommends reduced dimensions to save space without losing context (e.g., PCA from 3072 to 1024). [\[14\]](#) [\[15\]](#) [\[16\]](#) [\[17\]](#)

- **Static Evaluation Logic:** Heuristics (e.g., semantic search → 768D if volume > 1GB, based on variance retention > 95%).
- **Dynamic Evaluation Logic:** 3 iterations: Apply PCA on embedded samples, measure accuracy drop (ROUGE), adjust target dims (e.g., -128 if drop > 2%), balance space savings.
- **Iteration Logic:** Start at model default, reduce in steps, evaluate info loss, converge on minimal dims with < 2% accuracy drop.
- **Input/Output:** "recommended\_dims": 768, "analysis": {"savings": "67%", "accuracy\_impact": "1.5%"}.
- **Example:** For classification, static suggests 1024D (97% retention); dynamic tests 512-1536 on samples, iterates to 1024 for 98% ROUGE retention.

## 4. LLM Model for Inference API (POST /llm-model)

**Purpose:** Recommends LLMs (e.g., GPT-4, Llama-3) for RAG generation. [\[7\]](#) [\[8\]](#) [\[18\]](#)

- **Static Evaluation Logic:** Based on use case (e.g., complex queries → GPT-4 if accuracy high).
- **Dynamic Evaluation Logic:** 5 iterations: Generate responses on samples, score with ROUGE/BLEU/faithfulness, fine-tune params (e.g., temperature), select best.

- **Iteration Logic:** Baseline generation, metric eval, adjust (e.g., temp 0.7 → 0.5 if hallucinations), repeat.
- **Input/Output:** "recommended\_llm": "GPT-4o", "params": {"temperature": 0.7, "max\_tokens": 512}.
- **Example:** For customer support, static picks Llama-3 (cost-effective); dynamic iterates temp 0.5-1.0, selects 0.7 for 85% ROUGE.

## 5. Indexing API (POST /indexing)

**Purpose:** Recommends indexes (e.g., HNSW, IVFFlat) for DBs like MongoDB, Qdrant, Milvus, Redis, FAISS. [\[3\]](#) [\[4\]](#) [\[19\]](#) [\[20\]](#) [\[21\]](#)

- **Static Evaluation Logic:** Match to DB/scale (e.g., Qdrant → HNSW with M=16 if medium scale).
- **Dynamic Evaluation Logic:** 4 iterations: Index samples, query, measure recall/latency, tune ef\_search (e.g., +32 if recall <90%).
- **Iteration Logic:** Build index, test queries, adjust params, rebuild until optimal.
- **Input/Output:** "recommended\_index": "HNSW", "params": {"m": 16, "ef\_construct": 128}.
- **Example:** For Milvus, static suggests HNSW (ef=64); dynamic iterates ef 32-128, selects 64 for 95% recall.

## 6. Vector DB API (POST /vector-db)

**Purpose:** Recommends DB (e.g., Qdrant for medium scale). [\[3\]](#) [\[4\]](#) [\[19\]](#) [\[22\]](#)

- **Static Evaluation Logic:** Based on volume/privacy (e.g., high volume → Milvus if cloud OK).
- **Dynamic Evaluation Logic:** 3 iterations: Load samples into candidates, measure insert/query speed, select best.
- **Iteration Logic:** Benchmark subsets, scale simulation, refine.
- **Input/Output:** "recommended\_db": "Qdrant", "config": {...}.
- **Example:** For 10M vectors, static picks Milvus; dynamic tests insert time, confirms for <100ms latency.

## 7. Retrieval Mechanism API (POST /retrieval-mechanism)

**Purpose:** Recommends mechanisms (e.g., similarity search, hybrid) with params (top\_k, threshold). [\[8\]](#) [\[18\]](#) [\[23\]](#)

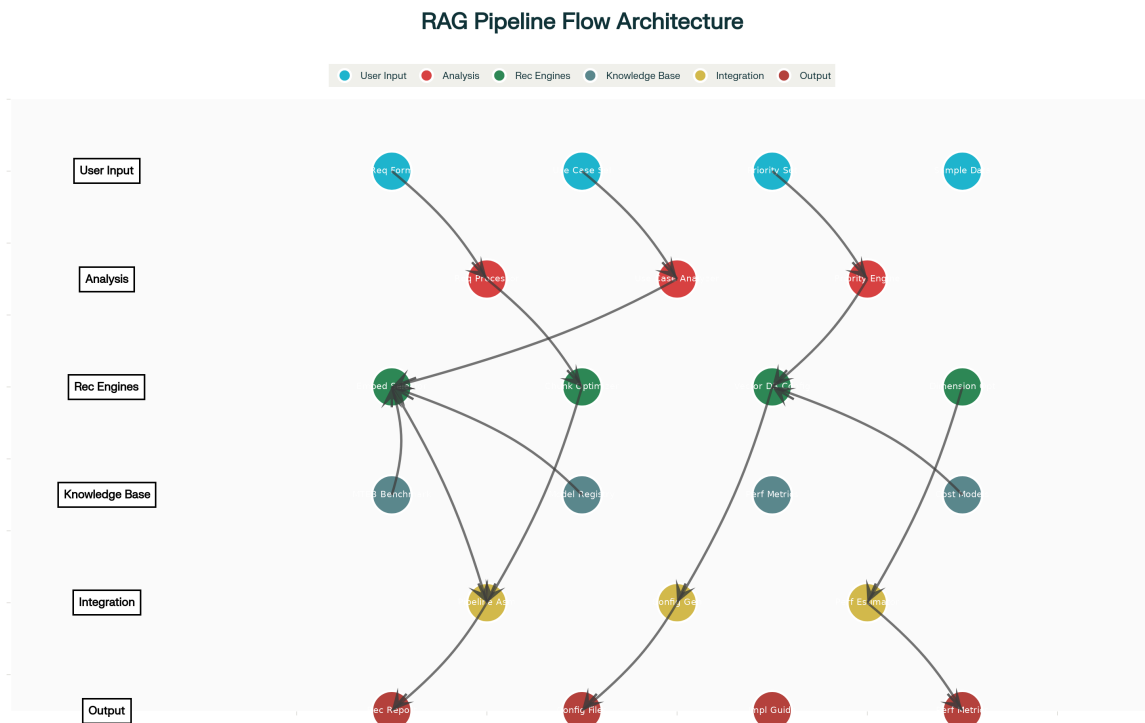
- **Static Evaluation Logic:** Use case-based (e.g., complex queries → hybrid with top\_k=5).
- **Dynamic Evaluation Logic:** 5 iterations: Retrieve on samples, measure precision@K/ROUGE, tune top\_k/threshold.
- **Iteration Logic:** Baseline retrieve, eval, adjust (e.g., top\_k +1 if precision low), converge.
- **Input/Output:** "mechanism": "hybrid", "params": {"top\_k": 5, "threshold": 0.7}.

- **Example:** For QA, static suggests similarity (top\_k=3); dynamic iterates to top\_k=5 for 92% precision.

## 8. Prompt Template API (POST /prompt-template)

**Purpose:** Recommends templates (e.g., few-shot for QA).

- **Static Evaluation Logic:** Map to use case (e.g., "Use context: {context} to answer: {query}").
- **Dynamic Evaluation Logic:** 4 iterations: Generate with variants, score BLEU/faithfulness, refine placeholders.
- **Iteration Logic:** Test templates, metric eval, mutate (e.g., add examples), select best.
- **Input/Output:** "template": "Context: {context}\nQuestion: {query}\nAnswer:", "rationale": "95% faithfulness".
- **Example:** For research, static suggests structured template; dynamic iterates additions, selects for 90% ROUGE.



RAG Pipeline Components Interaction Diagram

## Overarching Recommendation API (POST /complete-recommendation)

**Purpose:** Orchestrates the 8 APIs internally, runs iterations, and outputs a full pipeline.

- **Logic:** Call APIs sequentially/parallel, aggregate results, perform 3 global iterations (e.g., if overall ROUGE < threshold, re-optimize chunking/embedding).
- **Input:** evaluation\_id.

- **Output:** Integrated JSON with all recommendations, metrics, and pipeline config.
- **Example:** Combines outputs into {"embedding": "...", "chunking": "...", ..., "estimated\_performance": {"rouge": 0.88, "latency": "150ms"}}.

## Low-Level Design Diagrams with Evaluation Logic

Low-level designs include flowcharts for static/dynamic logic per API, with decision points for iterations.

## Evaluation Documentation and Reasoning

### Detailed Calculations:

- **Metrics Used:** ROUGE (semantic similarity), BLEU (exact match), Precision@K (retrieval), Faithfulness (hallucination check via entailment models), Latency (ms), Cost (\$/query).
- **Reasoning:** Static for quick starts (e.g., if priority=accuracy, weight MTEB 70%); Dynamic for precision (e.g., iterate until metric improvement <1%, using sample data subsets).
- **Iteration Example (General):** Iteration 1: Baseline params → Metrics calc (ROUGE=0.75). Iteration 2: Adjust (e.g., increase overlap) → Recalc (ROUGE=0.82). Stop if delta <0.02.

The complete redesigned documentation has been generated as a markdown file for implementation reference.

\*\*

1. <https://milvus.io/blog/how-to-choose-the-right-embedding-model.md>
2. <https://www.pinecone.io/learn/series/rag/embedding-models-rundown/>
3. <https://zilliz.com/learn/benchmark-vector-database-performance-techniques-and-insights>
4. <https://milvus.io/blog/benchmarks-lie-vector-dbs-deserve-a-real-test.md>
5. <https://milvus.io/blog/vdbbench-1-0-benchmarking-with-your-real-world-production-workloads.md>
6. <https://lakefs.io/blog/what-is-rag-pipeline/>
7. <https://developer.nvidia.com/blog/rag-101-demystifying-retrieval-augmented-generation-pipelines/>
8. <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/rag/rag-solution-design-and-evaluation-guide>
9. <https://sec.cloudapps.cisco.com/security/center/resources/selecting-embedding-models>
10. <https://www.pinecone.io/learn/chunking-strategies/>
11. <https://www.multimodal.dev/post/semantic-chunking-for-rag>
12. <https://community.databricks.com/t5/technical-blog/the-ultimate-guide-to-chunking-strategies-for-rag-applications/ba-p/113089>
13. <https://www.f22labs.com/blogs/7-chunking-strategies-in-rag-you-need-to-know/>
14. <https://devblogs.microsoft.com/azure-sql/embedding-models-and-dimensions-optimizing-the-performance-resource-usage-ratio/>
15. <https://openreview.net/forum?id=gDDW5zMKFe>
16. <https://arxiv.org/abs/2401.04408>

17. <https://milvus.io/ai-quick-reference/what-is-embedding-dimensionality-and-how-do-you-choose-it>
18. <https://humanloop.com/blog/rag-architectures>
19. <https://docs.mem0.ai/components/vector dbs/config>
20. <https://milvus.io/ai-quick-reference/what-are-the-key-configuration-parameters-for-an-hnsw-index-such-as-m-and-efconstructionefsearch-and-how-does-each-influence-the-tradeoff-between-index-size-build-time-query-speed-and-recall>
21. [https://docs.weaviate.io/academy/py/vector\\_index/hnsw](https://docs.weaviate.io/academy/py/vector_index/hnsw)
22. <https://www.instaclustr.com/education/vector-database/vector-databases-in-azure-6-service-options-and-how-to-get-started/>
23. <https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview>